

default



Memoria Caché

Anthony Almeida, Molina Jhanavi

2026-06-29

Outline

Índice

1. 2026-06-29 Memoria Caché	2
1.1. Preguntas y Términos clave (Cue):	2
1.2. Notas (Notes):	3
1.3. Resumen (Summary):	3
2. 2024-xx-xx Título de la clase/charla	3
2.1. Preguntas y Términos clave (Cue):	3
2.2. Notas (Notes):	3
2.3. Enlaces de Interés	4
2.4. Resumen (Summary):	4
3. 2026-06-27 ¿Qué pasa cuando la caché se llena? — Políticas de Caché	4
3.1. Preguntas y Términos clave (Cue):	4
3.2. Notas (Notes):	4
3.3. Resumen (Summary):	5
4. 2026-06-29 Métodos de Mapeo en Memoria Caché	5
4.1. Preguntas y Términos clave (Cue):	5
4.2. Notas (Notes):	6
4.3. Mapeo Directo	6
4.4. Mapeo Asociativo Puro	6
4.5. Mapeo Asociativo por Conjuntos	7
4.6. Tabla comparativa de los tres métodos	7
4.7. Resumen (Summary):	7
5. TODO Mis Tareas [50 %]	7
#+begincomment Este comentario contiene la estructura básica para toma de notas de acuerdo a la metodología explicada por la Universidad de Cornell.	

1. 2026-06-29 Memoria Caché

1.1. Preguntas y Términos clave (Cue):

término clave1

término clave2

1.2. Notas (Notes):

- nota1
- nota2
 - subnota
 - subnota
- nota3
 1. numerado
 2. numerado

1.3. Resumen (Summary):

Aquí se realiza un breve resumen `#+endcomment`

2. 2024-xx-xx Título de la clase/charla

2.1. Preguntas y Términos clave (Cue):

¿qué es el sistema de notas de U. Cornell?

TODO ????

2.2. Notas (Notes):

- Sistema eficiente de toma de apuntes
- Divide la hoja en tres secciones:
 1. Sección para preguntas: apuntar todo lo que no se entiende o la pregunta que surge de la charla
 2. *Sección para notas: se coloca las notas normales. Aquí podemos explotar Emacs y Programación literaria para incluir código de programación
 3. Resumen: Una vez terminada la clase, en una fase posterior y usando las notas tomadas, y resolviendo las inquietudes, se escribe un resumen de la clase.
- Emacs se puede usar para facilitar esta tarea dividiendo la pantalla mediante:
 - Horizontal: `C-x 2`
 - Vertical: `C-x 3`
 - Cambiar de ventana `C-x o`
 - Cerrar una ventana que ya no quiero `C-x 0`
 - Cerrar todas las demás ventanas excepto en la que el cursor está `C-x 1`
- Una desventaja de Emacs es que no podemos graficar en la pantalla
- Una ventaja es que podemos escribir la notación matemática usando $\LaTeX$
- Una ventaja es que podemos escribir el código de programación de cualquier lenguaje de programación y testearlo directamente desde el archivo.
 1. numerado

2. numerado

- Emacs nos permite adicionar el `TODO` para marcar las tareas y controlar cuáles ya están realizadas. Para esto generamos más viñetas con el `[ ]`. Si la tarea ya está realizada podemos dar `C-c C-c` en la línea y cambiar el estado de la tarea a realizada. Pruebe en el ejemplo.

2.3. Enlaces de Interés

- Página web de Cornell University sobre Toma de Notas
- YouTube

2.4. Resumen (Summary):

Emacs ofrece una herramienta que nos permite combinar código y literatura así como aprovechar $\text{\LaTeX}$ para escribir notación matemática. Mediante el uso de combinaciones de teclas se puede alternar entre distintas pantallas. Así por ejemplo, se puede dividir la pantalla en una región para apuntar las preguntas, otra para las notas y una tercera para el resumen. Algo que no incluye la estrategia de Cornell es apuntar las tareas. Pero esto lo podemos aprovechar con Emacs `TODO`.

3. 2026-06-27 ¿Qué pasa cuando la caché se llena? — Políticas de Caché

3.1. Preguntas y Términos clave (Cue):

¿Qué hago cuando no hay espacio en caché?

LRU saco lo que usé hace más tiempo

FIFO saco el que llegó primero, sin importar si lo uso

Aleatorio sorteo quién sale (funciona casi tan bien como LRU)

Write-Through actualizo caché y RAM al mismo tiempo

Write-Back actualizo la RAM solo cuando el dato sale de caché

Dirty bit marca que avisa "este dato fue modificado"

¿Caché dividida o unificada?

3.2. Notas (Notes):

$\llbracket$ label=sec:org2524a07 $\rrbracket$ ¿A quién saco cuando la caché está llena?

- Cuando llega un dato nuevo y no hay espacio, hay que expulsar uno. Eso lo decide el algoritmo de sustitución.

LRU saca el dato que lleva más tiempo sin usarse. Funciona bien porque si no lo usé recientemente, probablemente tampoco lo necesite pronto. Es el más usado en la práctica.

FIFO saca el que llegó primero, como la cola del banco. Problema: puede sacar datos que todavía se usan mucho.

Aleatorio elige al azar. Sorprendentemente casi tan bueno como LRU, con hardware mucho más simple.

$\llbracket$ label=sec:orgdf645ce $\rrbracket$ Ejercicio en vivo: FIFO con 3 marcos Secuencia: $1 \rightarrow 2 \rightarrow 3 \rightarrow 1 \rightarrow 4 \rightarrow 1 \rightarrow 2$

Ref.	Marco 1	Marco 2	Marco 3	¿Quién sale?	Resultado
1	1	—	—	nadie, hay espacio	Fallo
2	1	2	—	nadie, hay espacio	Fallo
3	1	2	3	nadie, hay espacio	Fallo
1	1	2	3	nadie, ya está	Acierto
4	4	2	3	sale el 1 (llegó primero)	Fallo
1	4	1	3	sale el 2	Fallo
2	4	1	2	sale el 3	Fallo

Resultado: 1 acierto de 7 referencias $\rightarrow$ Hit ratio 14%

Lección: FIFO expulsó al 1 justo después de usarlo (paso 4 $\rightarrow$ 5). LRU lo habría protegido y dado un acierto extra en el paso 6.

¿Cuándo escribimos los cambios en la RAM?

- Write-Through — escribo en caché y RAM al mismo tiempo:
 - Como corregir el borrador y el cuaderno oficial a la vez.
 - Ventaja: la RAM siempre está al día.
 - Desventaja: genera mucho tráfico constante.
- Write-Back — escribo en la RAM solo cuando el dato sale de caché:
 - El dato modificado lleva un dirty bit (.está sucio, hay que pasarlo en limpio"). Solo se copia a RAM al ser expulsado.
 - Ventaja: mucho menos tráfico, mejor rendimiento.
 - Desventaja: la RAM puede tener datos viejos por un tiempo.
- ¿Cuándo conviene cada uno? Si un bloque se modifica más de 8 veces antes de salir de caché, Write-Back acumula menos tiempo de escritura y gana.

Dos detalles más de diseño

- Tamaño de bloque: el bloque ideal tiene entre 8 y 64 bytes. Muy pequeño aprovecha poco la localidad espacial; muy grande tarda más en cargarse y trae datos que quizás no uses. El estándar actual es 64 bytes.
- Caché dividida (split): separar en I-cache (instrucciones) y D-cache (datos) elimina la competencia cuando el CPU necesita buscar la próxima instrucción y leer un dato al mismo tiempo. Es el estándar en L1 de todos los procesadores modernos.

3.3. Resumen (Summary):

Cuando la caché se llena, LRU expulsa lo menos usado recientemente y es la mejor opción; FIFO expulsa lo más antiguo sin importar su uso (puede tomar malas decisiones, como vimos en el ejercicio). Para las escrituras, Write-Through mantiene la RAM siempre actualizada pero genera tráfico; Write-Back es más eficiente pero la RAM puede quedar desactualizada temporalmente. El bloque estándar mide 64 bytes y la L1 moderna siempre está dividida en instrucciones y datos.

4. 2026-06-29 Métodos de Mapeo en Memoria Caché

4.1. Preguntas y Términos clave (Cue):

¿Cómo decide la caché dónde guardar un bloque?

Mapeo Directo: cada bloque de RAM tiene una única línea asignada en caché

Thrashing: fallo continuo cuando dos bloques compiten por la misma línea

Tag (etiqueta): bits que identifican de qué bloque de RAM viene el dato

Mapeo Asociativo Puro: cualquier bloque puede guardarse en cualquier línea libre

CAM: hardware que compara etiquetas en paralelo en todas las líneas simultáneamente

Mapeo Asociativo por Conjuntos: el bloque tiene conjunto fijo pero línea libre dentro de él

K-Way: número de líneas por conjunto (ej. 4-way, 8-way)

¿Por qué el asociativo por conjuntos es el estándar de la industria?

4.2. Notas (Notes):

4.3. Mapeo Directo

- Cada bloque de la RAM tiene una única línea posible en caché, calculada así:
$$\text{línea} = \text{número}_{\text{de bloque}} \bmod \text{número}_{\text{de líneas en caché}}$$
- Ejemplo con 16 384 líneas: el Bloque 0 y el Bloque 16 384 comparten siempre la Línea 0, nunca pueden coexistir en caché al mismo tiempo.
- Ventaja: hardware extremadamente simple y barato; la dirección ya dice exactamente adónde ir, sin búsqueda.
- Desventaja: Thrashing ,si el programa alterna entre dos bloques que comparten línea, la caché los borra y recarga indefinidamente, destruyendo el rendimiento.
- La dirección de 32 bits se divide en tres campos:

Campo	Bits	Función
Etiqueta	8	Identifica de qué bloque de RAM viene el dato
Línea	14	Indica directamente a qué fila de caché ir
Palabra	10	Selecciona el byte exacto dentro del bloque

4.4. Mapeo Asociativo Puro

- Flexibilidad total: cualquier bloque de RAM puede ocupar cualquier línea libre de la caché, sin restricción matemática de ubicación.
- Ventaja: elimina por completo el Thrashing, el Bloque 0 y el Bloque 16 384 conviven sin problema en líneas distintas.
- Desventaja: como el bloque puede estar en cualquier lugar, para encontrarlo hay que revisar todas las líneas al mismo tiempo.
 - Esto requiere hardware especialísimo: la Memoria CAM (Content Addressable Memory), que compara la etiqueta buscada contra todas las líneas en paralelo y en tiempo real.
 - Es extremadamente cara, consume mucha energía y es compleja de fabricar en silicio.
- La dirección de 32 bits pierde el campo de línea:

Campo	Bits	Función
Etiqueta	22	La CAM la compara en paralelo con toda la caché
Palabra	10	Selecciona el byte exacto dentro del bloque

- Por su costo, la asociatividad pura solo se usa en cachés muy pequeñas dentro de los procesadores reales.

4.5. Mapeo Asociativo por Conjuntos

- Combina lo mejor de los dos métodos anteriores, es el estándar en casi todos los procesadores modernos.
- La caché se divide en conjuntos, cada bloque tiene un conjunto obligatorio (como en el directo) pero dentro de ese conjunto puede ocupar cualquier línea libre (como en el asociativo puro).
- El número de líneas por conjunto se llama K-Way:
 - 2-way: 2 líneas por conjunto
 - 4-way / 8-way: lo más común en cachés L1 y L2 de Intel y AMD
- Ventaja: elimina casi por completo el Thrashing, dos bloques que caen en el mismo conjunto simplemente ocupan líneas distintas dentro de él y conviven sin problemas.
- Ventaja: el circuito de comparación es pequeño (solo compara las pocas líneas del conjunto), así que es rápido y económico.
- Desventaja: requiere algo más de lógica de control que el mapeo directo, aunque la diferencia es mínima en la práctica.
- La dirección de 32 bits se divide en:

Campo	Bits	Función
Etiqueta	9	Confirma Hit o Miss comparando solo las líneas del conjunto
Conjunto	13	Lleva al procesador directo al grupo que le corresponde
Palabra	10	Selecciona el byte exacto dentro del bloque

4.6. Tabla comparativa de los tres métodos

Criterio	Mapeo Directo	Asociativo Puro	Asoc. por Conjuntos
Flexibilidad	Ninguna	Total	Controlada (media)
Riesgo Thrashing	Muy alto	Nulo	Muy bajo
Costo hardware	Muy bajo	Muy alto (requiere CAM)	Moderado
Uso real	Raro	Solo cachés muy pequeñas	Estándar en L1 y L2

Conclusión: el Asociativo por Conjuntos gana porque da flexibilidad suficiente para evitar el Thrashing sin encarecer el chip. Más de 8 o 16 vías ya no vale la pena: el circuito se vuelve caro sin ganar rendimiento real (Stallings).

4.7. Resumen (Summary):

Existen tres métodos para decidir dónde guarda la caché un bloque de RAM. El Mapeo Directo asigna a cada bloque una única línea posible mediante el residuo de la división, es barato pero sufre Thrashing cuando dos bloques compiten por el mismo espacio, el Mapeo Asociativo Puro permite guardar cualquier bloque en cualquier línea libre, eliminando el Thrashing por completo, pero exige hardware CAM muy costoso para buscar en paralelo, el Mapeo Asociativo por Conjuntos combina ambos: el bloque tiene un conjunto fijo pero elige libremente entre las líneas de ese conjunto (K-Way), esto reduce el Thrashing casi a cero con un costo de hardware moderado, razón por la que es el estándar en las cachés L1 y L2 de todos los procesadores modernos.

5. TODO Mis Tareas [50 %]

☒ Leer este documento

☐ Escribir el resumen de la exposición 1